Notes for C++

Section 10 : Text Files

Using files

Data in variables in a program is stored in the computer memory (RAM). However RAM is **volatile** – i.e. the data in it disappears when the power is switched off (or fails). If we want to keep data permanently we need to store it on a **storage device**. Storage devices include hard discs, optical storage devices (DVD) and flash memory. Many hard discs are now Solid State Drives (SSD) which is actually a type of flash memory. Devices have different access speeds: RAM is fastest, then flash memory, then hard discs, then optical devices. It will take longer to read data from a storage device than from RAM. For this reason, we often transfer data to RAM (put it into variables) before we use it. A device (such as a hard disc) often appears faster than it actually is because data is transferred to memory and used from there. While this increases speed it is important to realise that data is not actually on the disc, but is in volatile memory.

We store data in **files**. We write a program to load the data into the file (this is called ***writing*** to the file). Alternatively we can use other programs (e.g. an editor) to write data to the file. We can then write other programs to look at the data in the file (this is called ***reading*** the file). The data in the file is permanent and can be used many times after the file has been set up. Every file has a name. The name can include the drive, the path, the file name and the extension.

For example: `F:\University\159101\Program1.cpp` (use / in Linux)

Let us look at the various parts of the file name:

`F:` this is the **drive letter**. Each device has a letter, e.g. the hard disc on a home PC is usually C: Different **partitions** on a hard disc may also have different drive letters. If you are using a file on a storage device (e.g. flash memory) ensure that you have the correct drive letter for that device. What drives do the letters A: and B: stand for? More information about drive letters can be found at http://en.wikipedia.org/wiki/Drive_letter_assignment

`\University\159101\` this is the **path** (or the list of folders). In this example, there is a folder called `University` and inside that folder there is another folder called `159101`.

`Program1` this is the **file name**. Use meaningful names so that you just have to look at the name and it tells you what is stored inside the file. File names must not contain strange characters such as a dot or a comma or similar.

`.cpp` this is the **extension**. The extension tells us the type of the file, e.g. this file is a C++ program. Examples of extensions include: `.docx .txt .jpg .cpp .java .xlsx .exe` People keep inventing extensions – that is why there are such a lot of them!

There is a short cut that we often use that avoids all these complications with file names. When using a file from inside a program, you do not have to supply the drive letter and the path as long as the file is in the same directory (or folder) that you are working in. For example, if we were using the file given above and were working inside the same folder we could then use the name `Program1.c` and we would not have to provide the drive letter and the path.

Unfortunately, certain operating systems (especially Windows) try to hide the file extensions. Always change the folder options so that you can see the extensions as they are important information. In

159.101 we will be using the extension `.txt` because we will mainly read and write text files. However text files can have other extensions and, in fact, you can invent your own extensions!

Text files

There are many different kinds of files. In our course we will use only at text files. **Text files consist of lines of simple text.** There are no fonts (e.g. italics) in a text file. Any program you have written is a text file as it consists of lines of text. However, jpg files or Word files are not text files as are encoded in a particular way. See what happens in you open a jpg file with and editor such as Wordpad. Note that text files may have various extensions, e.g. .cpp and .txt are both extensions that are used for text files.

Using text files from a C++ program

The basic rules for using a file from your own program are:

- Know the full name of the file
- Open the file
- Access the file (reading or writing)
- Close the file

A light-hearted reminder of this process can be found here:
https://www.youtube.com/watch?v=-zQgEqDdvuo

Remember that you can always read or write to a text file using any editor such as Wordpad, etc.

A useful cheat about entering file names

As describes above, a file name can be rather lengthy and have many sections – the drive letter, the path, etc. Sometimes it is necessary to enter the FULL file name as discussed in this section. However, after a while you will notice that our examples use much simpler file names such as Names.txt. The drive letter, the path, etc are all missing – we use only the file name and the extension.

This shortcut works ONLY IF the file is in the same folder as your program. As long as you put the data file in the same folder as the program, then you can leave out the drive letter and the path.

When entering a file name, you have a choice:

a) Enter the FULL file name including drive letter, path, etc
b) Enter only the file name and extension BUT the data file must be in the same folder as the program.

A program to load lines of text into a text file (writing to the file)

This program demonstrates how to create a new file and load data into the file – known as ***writing*** to the file. We will load four lines of text into the file.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
fstream datafile;          // define a file object
string filename, text;

int main() {
  // 1. Load the full name of the file into a string
  filename = "F:\\TESTING\\data.txt";   /* use \\ for \ */

  // 2. Open the file for writing (output)
  datafile.open(filename.c_str(), fstream::out);

  // 3. Access the file - in this case, write to the file
  cout << "Now writing to " << filename << endl;
  text = "This is the first line in the file";
  datafile << text << endl;
  text = "This is the second line in the file";
  datafile << text << endl;
  text = "And another line in the file...";
  datafile << text << endl;
  text = "This is the last line in the file";
  datafile << text << endl;
  cout << "Completed writing to file.\n";

  // 4. Close the file
  datafile.close();
}
```

We have introduced several new concepts in this program. Here is an overview:

`#include <fstream>`
We include the library `fstream` when working with files. The library `iostream` provides functions to move data to and from IO (input-output) devices and `fstream` provides functions to move data to and from files.

`fstream datafile;`
This line defines a file object and gives it a name. You can use any name (similar to variable names). `datafile` is an object (more on objects later) and is used to let the system know which file you are working with.

`filename = "F:\\TESTING\\data.txt";`
Load the name of the file into a string variable. Note that you need \\ (special character) to indicate a single backslash. **WARNING:** this program will only run if the USB is drive F: and there is already a folder called TESTING on the USB. Set this up before running the program. The easiest way to sort out the file name is to **have only the file name and extension** (no drive letter and no path) and then the text file will appear in the folder with your program.

`datafile.open(filename.c_str(), fstream::out);`
Open the file for writing (output). This line connects the file object (datafile) with the file name. The first parameter must be the "C string version" of the filename, which is why we show the filename as `filename.c_str()`. The second parameter informs the system if the file is used for writing (output) or reading (input) or something else – there are some other alternatives.

`cout << "Now writing to " << filename << endl;`

This line shows a message on the screen. This is not needed but many people get anxious if the run an application and it shows a blank screen. When designing a program that would otherwise have a blank screen, it is a good idea to display a message to let people know what is going on.

`datafile << text << endl;`
This line loads text to the file using the file object. You write the instruction following all the rules for `cout` but you use the name of the file object instead of `cout`.

`datafile.close();`
This line closes the file. The file object `datafile` cannot be used again in the program after this function call.

This program will ***create a new file***. Before you run the program there is not a file called `data.txt`. If the program runs successfully, the file will appear.

IMPORTANT – after running a program to load data into a file, you should always check that the file has appeared. It may be on a USB drive (depends on the drive letter). Also look at the file itself to see if the lines of text were output correctly. You can look at a file by using and text editor such as Wordpad, etc.

PRACTICE – you must practice writing text to files. One exercise is to select an old (simple) program and convert it to output line to a file instead of to `cout`.

A program to append lines of text at the end of a text file (appending)

Files often need to be edited. We will take the file we created in the previous program and we will include two new lines at the end of the file – adding new lines at the end is called **appending**.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream myfile;              // define a file object
string filename, text;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for both output and appending
  myfile.open(filename.c_str(), fstream::out | fstream::app);

  // 3. Access the file - in this case, write to the file
  cout << "Now appending lines to " << filename << endl;
  text = "This is a new line to be appended";
  myfile << text << endl;
  text = "This is another new line to be appended";
  myfile << text << endl;
  cout << "Completed writing to file.\n";

  // 4. Close the file
```

```
  myfile.close();
}
```

Notes about this program:

`fstream myfile;`
In this program I decided to use the name `myfile` for the file object. You can use whatever name you want to but get into the good habit of using meaningful names.

`// 1. The user will enter the full name of the file`
In this program, the user has to enter the file name. There are two things the user has to get right:
a) make sure you know the EXACT file name before entering it
b) do not use the \\ when entering data (only use \\ in the code). So in this case the correct file name to be entered would be `F:\TESTING\data.txt`

`myfile.open(filename.c_str(), fstream::out | fstream::app);`
The file is opened for both output and appending. Because you have informed the system that you are appending, any output to the file will be added after the existing lines in the file.

After the program has run, **CHECK THE FILE**. Use an editor to open the file and check the the new lines have been appended at the end.

A program to load lines of text from a file (reading from the file)

The previous example covered loading data from a program into a text file (writing). We will now move on to loading data from a text file into a program (reading).

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int number;
fstream inputfile;          // define a file object
string filename, line;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  inputfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (inputfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {

    // 3. Access the file - read all the lines from the file
    cout << "Displaying all the lines from " << filename << endl;
    number = 0;
    while (getline(inputfile, line)) {
```

```
      number++;
      cout << number << ": " << line << endl;
    }
    cout << "Displayed " << number << " lines from the file.\n";

    // 4. Close the file
    inputfile.close();
  }
}
```

Notes about this program:

`// 1. The user will enter the full name of the file`
You can use this program to read any file you like. Before entering the file name:
a) make sure you know the EXACT file name before entering it
b) make sure the USB is plugged in (and what drive letter it is) – if you are using the USB
c) do not use the \\ when entering data (only use \\ in the code), e.g. `F:\TESTING\data.txt`

`inputfile.open(filename.c_str(), fstream::in);`
Open the file for reading (input).

`if (inputfile.is_open() == false) {`
The opening of the file may fail for a variety of reasons, e.g. you may have entered the incorrect name, the USB may not be plugged in, you included an incorrect folder name in the path, etc. It is important to use the `is_open` function to check if a file has opened successfully. If it has not opened, print an error message alerting the user to that fact that the file could not be opened. This check is only done when reading from a file. When you are writing to a file, a new file will be created anyway.

`while (getline(inputfile, line)) {`
We set up a loop to read every line from the file. It is a while loop because we do not know how many lines there will be. We have already used `getline` with `cin`, now we use it with the file object.

`number++;`
I included a line number variable to count how many lines I had read from the file. This is entirely optional and you do not have to do this unless you actually need the line numbers.

Reading one word at a time from a text file

Sometimes we want to read one word at a time from a text file. Reasons for doing this include:

a) counting words – students are often told "write an essay of 4,000 words" or similar. It is useful to write programs that read in the text file and counts the number of words in the file. People make money by selling programs to count words – google "word count" to find out more!

b) analysing text – people study books and documents to look for patterns, word use, etc. Once again, google "analyse text" to find out more about this.

c) compiling computer programs – a program written in C (or any other language) cannot be executed on a computer. It first needs to be converted to machine code. This process is carried out by a compiler – a program that reads other programs and checks every instruction.

A program to read one word at a time from a text file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int wordcount;
fstream datafile;          // define a file object
string filename, word;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  datafile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (datafile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {

    // 3. Access the file - read one word at a time from the file
    cout << "Displaying all the words from " << filename << endl;
    wordcount = 0;
    while (datafile >> word) {
      wordcount++;
      cout << word << endl;
    }
    cout << "There are " << wordcount << " words in the file.\n";

    // 4. Close the file
    datafile.close();
  }
}
```

Interesting things to notice in this program:

- as usual, only run the program if the datafile has opened successfully
- use `datafile >> word` to read one word at a time from the file
- the while loop will keep going until the file runs out
- look at the display – are these words actually "words" or not...? Note that the punctuation (quotes, semi-colons, etc) all appear as part of the words.

A program to count if-statements in another program

A compiler is a program that reads in another program and checks every statement.
We are not going to write a compiler but it is interesting to get an idea of what a compiler does.
Write a program to read in another program and look for the if statements in the other program.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
int ifcount;
fstream programfile;          // define a file object
string filename, str;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the program: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  programfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (programfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {

    // 3. Access the file - read one word at a time from the file
    ifcount = 0;
    while (programfile >> str) {
      if (str == "if") {
        ifcount++;
      }
    }
    cout << "There are " << ifcount << " if statements.\n";

    // 4. Close the file
    programfile.close();
  }
}
```

WARNING: this program has a ***logic error*** – it runs but sometimes produces an incorrect count. For example, if you run it and look through the program listed above (i.e. the program looks at itself!) it will count 4 if statements but there are actually only 2 if statements. Can you see why?

A logic error is the worst kind of error because the program sometimes produces the correct result and sometimes not. The only way to find a logic error is by extensive testing.

Challenge 1: fix the program to accurately count if statements

Challenge 2: modify the program to display line numbers, i.e. for each if statement it should display the line number of that particular if statement.

Read one LINE at a time or one WORD at a time?

Creating a program is all about making decisions. For example, when to use a while loop and when to use a for loop? Should I use a function or not? Should I use local variables or not? Similarly, when reading data from a file you need to decide if you are going to read **one line at a time** or **one word at a time**.

Saving lines of data using a standard template

Text files can be used to save data where each line in the file represents a data entry and a standard pattern (or template) is used for each line.

For example, we may want to save information about books and we could define the template to be:
Title;Author;Year

An example of one entry (for one book) in our file could be a line that looks like this:
The Two Towers;Tolkien J.R.R.;1954

Each line in the file saves the title of the book, the author of the book, and the year in which the book was first published. You could easily expand the system to save more information if you want to.

Note that each item on the line is separated by a semi-colon. These are called ***separators*** or ***delimiters***. A separator can be anything you like so long as it does not normally appear in the title or author of a book. Finding a good separator is similar to finding a good trailing value – it must not be legal data.

We can now write a program that reads data from the file and checks the data in some way. Examples of typical checks could be – how many books are there in the file? - how many books were published before 1999? - how many books are written by a particular author? - how many books have the word "computer" in the title? etc, etc. There are lots of possible checks that can be done.

How do we approach this program? Should we read one line at a time or one word at a time?
- reading one line at a time means we don't separate the title, author and year
- reading one word at a time means we get "words" such as "Towers;Tolkien"

We will take the approach of reading one line at a time and then breaking the line into the separate sections. This does not mean this approach is better – it is just one possible alternative.

A program to find books written by a particular author

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void breakstring(string s);

int count;
fstream bookfile;          // define a file object
string filename, searchname, line, title, author, yearstring;

int main() {
  // 1. The user will enter the full name of the file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;
  getchar();  // clear the input buffer before reading the author

  // 2. Open the file for reading (input)
  bookfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (bookfile.is_open() == false) {
```

```
    cout << "ERROR: not able to open " << filename << endl;
  } else {
    // 3. Enter the name of the author you want
    cout << "Enter the name of the author: ";
    getline (cin, searchname);   // names can have spaces
    cout << "Here is the list of books written by ";
    cout << searchname << endl;

    // 4. Access the file - read all the lines from the file
    count = 0;
    while (getline(bookfile, line)) {
      cout << line << endl;  // DEBUG cout to display all lines
      breakstring(line);
      // convert yearstring to a number
      if (author == searchname) {
        cout << yearstring << " " << title << endl;
        count++;
      }
    }
    cout << searchname << " wrote " << count << " books.\n";

    // 5. Close the file
    bookfile.close();
  }
}

void breakstring(string s) {
  // break the string s into smaller strings using the separator
  int total, i;

  i = 0;
  title = "";
  while (s[i] != ';') {
    title = title + s[i];
    i++;
  }
  i++;  // jump over the semi-colon
  author = "";
  while (s[i] != ';') {
    author = author + s[i];
    i++;
  }
  i++;  // jump over the semi-colon
  yearstring = "";
  total = s.length();
  while (i < total) {
    yearstring = yearstring + s[i];
    i++;
  }
}
```

Interesting things to notice in this program:

- before running the program, make a text file with some lines that follow the pattern
- we need to clear the input buffer before reading the name of the author

- the author name must be read as a line of text because names often have spaces in them
- it is useful to display lines as you read them to check that the file is correct – you can remove this debug cout when the program is working.
- the breakstring function is used to help with program design – it keeps main shorter
- the function is not "stand alone". It uses strings that are global variables because a function can only return one answer and we want 3 answers (the 3 short strings).
- yearstring is a string. It is not a number. You need to convert it to a number if, for example, you wanted to check which books were published before 1999.
- are there other ways to write this program? Definitely, but it is good to get used to the fact that strings are arrays of characters.
- note the extra i++ in the function to jump over the semi-colons.
- the last loop in the function does not look for a semi-colon. It runs to the end of the string.
- the function has a logic error. It works fine most of the time but there is one situation when it displays the wrong information. What situation is this and how would you fix it?

Can we access more than one file in a program?

Yes we can. You can open as many files as you like. Make sure you have a file variable for each file that you are using and that you close all the files. Here is an example...

Program to read names from a file and separate A – L from M – Z

There is a text file called Names.txt which has one name (perhaps with spaces) on each line of the file. The file is not sorted. We want to read all the names from the file and create two new files where one of the files contains the names from A to L and the other file contains the names from M to Z.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream fileone, filetwo, filethree;   // define three file objects
string filename, name;

int main() {
  // 1. Open the first file for reading
  filename = "Names.txt";
  fileone.open(filename.c_str(), fstream::in);

  // 2. Open the other two files for writing
  filename = "AtoL.txt";
  filetwo.open(filename.c_str(), fstream::out);
  filename = "MtoZ.txt";
  filethree.open(filename.c_str(), fstream::out);

  // only continue if the file has opened successfully
  if (fileone.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {
    // 3. Access the file - read all the lines from the file
    cout << "Processing the names to create new files...\n";
```

```
    while (getline(fileone, name)) {
      cout << name << endl;  // DEBUG cout to display all lines
      if ((name[0] >= 'A') && (name[0] <= 'L')) {
        filetwo << name << endl;
      } else {
        filethree << name << endl;
      }
    }
    cout << "File processing complete, check the new files.\n";

    // 4. Close all the files
    fileone.close();
    filetwo.close();
    filethree.close();
  }
}
```

Transferring data between your program and a standard file format

It is useful to be able to transfer data between our own programs and standard packages. Many packages are set up to be able to do this. For example, spreadsheets such as Excel are usually set up to read data from or write data to simple text files called 'comma-separated value' (csv) files. A csv file has a line of entries (numbers or strings) separated by commas.

Program to write data into a csv file

Students have written two tests. Read the student names and the two marks from the screen and write the data into a csv file. Then open the csv file with a spreadsheet.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream csvfile;          // define a file object
int mark1, mark2;
string filename, name;

int main() {
  // 1. open the file
  filename = "Test-results.csv";
  csvfile.open(filename.c_str(), fstream::out);

  // 2. enter the data using a trailing value of ?? for the name
  cout << "Enter the student name and end with ?? ";
  getline (cin, name);  // names may have spaces
  while (name != "??") {
    cout << "Enter the result for test 1: ";
    cin >> mark1;
    cout << "Enter the result for test 2: ";
    cin >> mark2;
    getchar();            // clear the input buffer
    csvfile << name << "," << mark1 << "," << mark2 << endl;
```

```
    cout << "Enter the student name and end with ?? ";
    getline (cin, name);  // names may have spaces
  }
  cout << "You have created the file called " << filename << endl;
  // 4. Close the file
  csvfile.close();
}
```

Two ways of opening a csv file

Option 1: Open the file as a spreadsheet. Usually this is done by double-clicking on the file (it must have the extension .csv). You will not see the commas in the spreadsheet.

Option 2: Open the file as a text file. Open the file through an editor such as gcc, notepad, etc. Run the editor first, then select open, then select the file. You will see the commas in the csv file.

Program to read data from a csv file

Make a spreadsheet where column 1 is an ID number and column 2 is an exam result, for example:
```
6847   53
```
Save the spreadsheet as a csv file. You do this by selecting "Save As" and then changing the type of the file to "CSV (Comma delimited) (*.csv)". There are several types of csv file and the other types may not work correctly.

Note that the numbers in the csv file are saved on one line with a comma between them. Write a program that reads from the csv file into corresponding arrays (ID and mark). Calculate the average and print all IDs where the exam result is higher than the average.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

fstream csvfile;          // define a file object
int id[100], exam[100], count, i;
float average, total;
string filename;
char comma;

int main() {
  // 1. The user will enter the full name of the csv file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  csvfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (csvfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {
    // 3. Read the data one number at a time
    i = 0;
```

```
    total = 0.0;
    // read lines from the file - note how we read the comma
    while (csvfile >> id[i] >> comma >> exam[i]) {
      total = total + exam[i];
      i++;
    }
    // 4. Close the file
    csvfile.close();

    // 5. Calculate and display the results
    count = i;
    average = total / count;
    cout << "The class average is " << average << endl;
    for (i = 0; i < count; i++) {
      if (exam[i] > average) {
        cout << "Student " << id[i] << " achieved ";
        cout << exam[i] << endl;
      }
    }
  }
}
```

Things that can go wrong with this program

The biggest problem is not actually the program but the creation of the csv file. Depending on your version of Excel, and various other factors, you may get invisible characters embedded in your csv file. If this happens, the program will not produce the correct output. If this happens, use the version of the program below which reads each line as a string and then extracts the numbers.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// this version of the program reads lines of text
// and then converts to numbers

fstream csvfile;          // define a file object
int id[100], exam[100], count, num, i, k;
float average, total;
string filename, line;;

int main() {
  // 1. The user will enter the full name of the csv file
  cout << "Enter the FULL name of the file: ";
  cin >> filename;

  // 2. Open the file for reading (input)
  csvfile.open(filename.c_str(), fstream::in);

  // only continue if the file has opened successfully
  if (csvfile.is_open() == false) {
    cout << "ERROR: not able to open " << filename << endl;
  } else {
```

```
    // 3. Read lines of text and convert to numbers
    i = 0;
    total = 0.0;
    while (getline(csvfile, line)) {
      cout << line << endl; // debug statement
      line = line + "$";  // use a $ to mark the end of the line
      k = 0;
      num = 0;
      while (line[k] != ',') {
        num = num * 10 + (line[k] - 48);
        k++;
      }
      id[i] = num;
      k++;  // skip over the comma
      num = 0;
      while (line[k] != '$') {
        num = num * 10 + (line[k] - 48);
        k++;
      }
      exam[i] = num;
      total = total + exam[i];
      i++;
    }
    // 4. Close the file
    csvfile.close();

    // 5. Calculate and display the results
    count = i;
    average = total / count;
    cout << "The class average is " << average << endl;
    for (i = 0; i < count; i++) {
      if (exam[i] > average) {
        cout << "Student " << id[i] << " achieved ";
        cout << exam[i] << endl;
      }
    }
  }
}
```